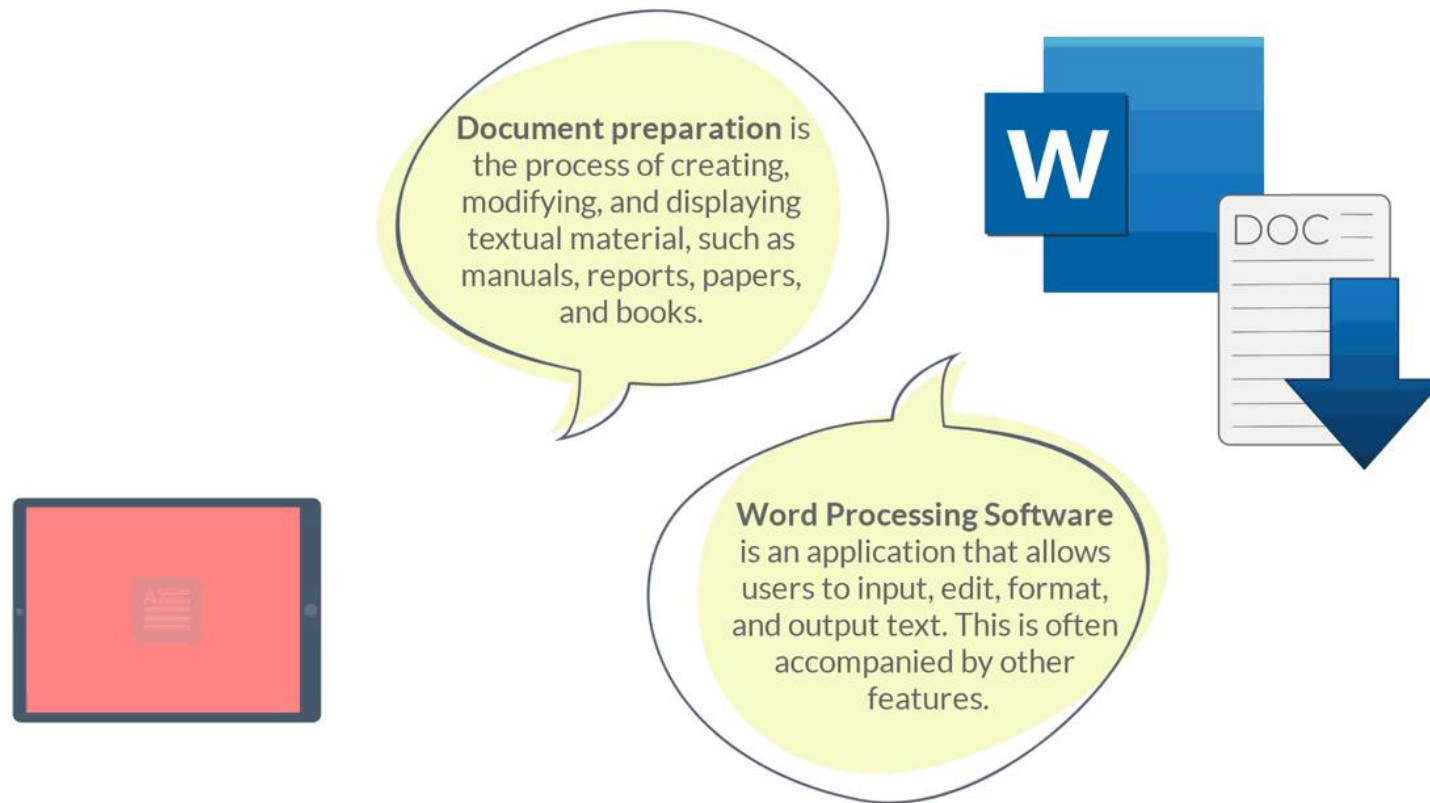


Unit-2 Word Level Processing

- Types of Morphology: English and Indian Languages,
- Finite-State Morphological Parsing,
- building a Finite-State Lexicon,
- Finite-State Transducers,
- FSTs for Morphological Parsing,
- Transducers and Orthographic rules,
- The Porter Stemmer,
- Word and Sentence Tokenization,
- Detecting and Correcting Spelling Errors,
- Minimum Edit Distance,
- Human Morphological Processing
- N-Grams: Building N-gram for spelling corrections, N-gram for language model.

Unit-2 Word Level Processing

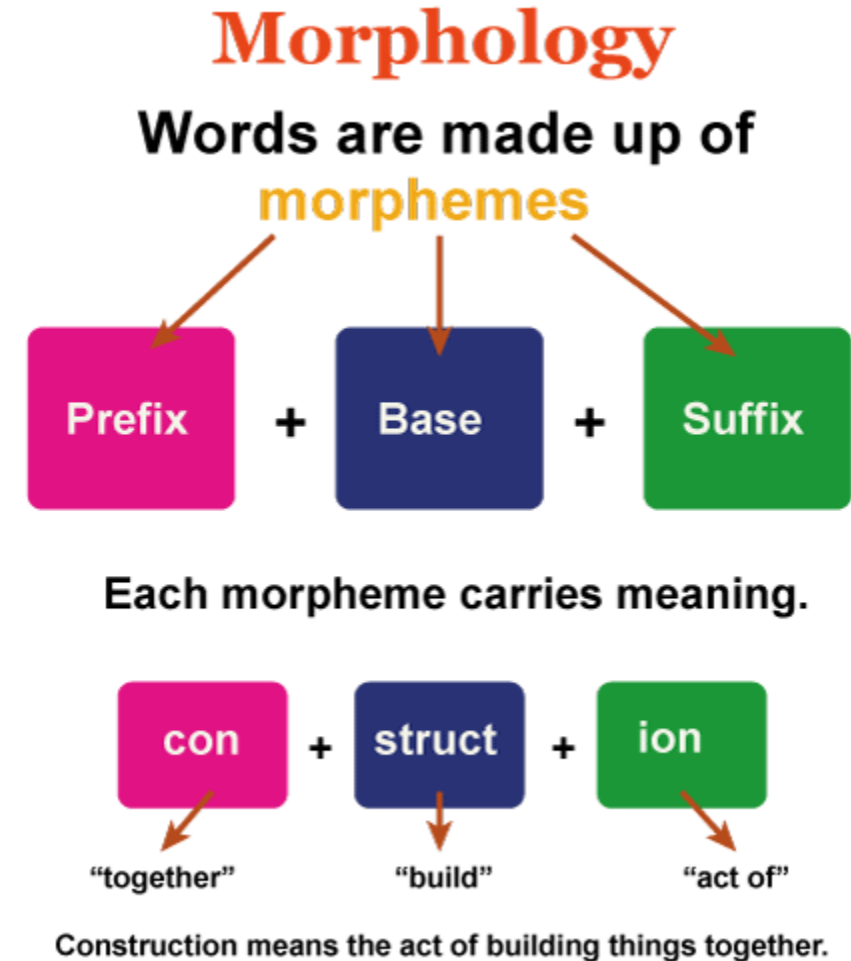


➤ Types of Morphology: English and Indian Languages,

Morphology is a branch of linguistics that focuses on the way in which words are formed from morphemes.

There are two types of morphemes namely

- lexical morphemes and
- grammatical morphemes



Unit-2 Word Level Processing

- Regular Expressions
- A regular expression (RE) is a language for specifying text search strings. RE helps us to match or find other strings or sets of strings, using a specialized syntax held in a pattern.
- Regular expressions are used to search texts in UNIX as well as in MS WORD in identical way.

Properties of Regular Expressions

- Followings are some of the important properties of RE–

- American Mathematician **Stephen Cole Kleene** formalized the **Regular Expression language**.
- RE is a **formula in a special language**, which can be used for specifying **simple classes of strings, a sequence of symbols**.
- In other **words**, we can say that RE is an **algebraic notation** for **characterizing a set of strings**.
- Regular expression requires two things, one is the **pattern** that we wish to search and other is a **corpus of text** from which we need to search.

Properties of Regular Expressions

- Mathematically, A Regular Expression can be defined as follows –
- ϵ is a Regular Expression, which indicates that the language is having an empty string.
- ϕ is a Regular Expression which denotes that it is an empty language.
- If X and Y are Regular Expressions, then
 - X, Y
 - $X.Y$ (Concatenation of XY)
 - $X+Y$ (Union of X and Y)
 - X^*, Y^* (Kleen Closure of X and Y)
- are also regular expressions.
- If a string is derived from above rules then that would also be a regular expression.

Examples of Regular Expressions

Regular Expressions	Regular Set
$(0 + 10^*)$	$\{0, 1, 10, 100, 1000, 10000, \dots\}$
(0^*10^*)	$\{1, 01, 10, 010, 0010, \dots\}$
$(0 + \varepsilon)(1 + \varepsilon)$	$\{\varepsilon, 0, 1, 01\}$
$(a+b)^*$	It would be set of strings of a's and b's of any length which also includes the null string i.e. $\{\varepsilon, a, b, aa, ab, bb, ba, aaa, \dots\}$
$(a+b)^*abb$	It would be set of strings of a's and b's ending with the string abb i.e. $\{abb, aabb, babb, aaabb, ababb, \dots\}$
$(11)^*$	It would be set consisting of even number of 1's which also includes an empty string i.e. $\{\varepsilon, 11, 1111, 111111, \dots\}$
$(aa)^*(bb)^*b$	It would be set of strings consisting of even number of a's followed by odd number of b's i.e. $\{b, aab, aabbb, aabbbbb, aaaab, aaaabbb, \dots\}$
$(aa + ab + ba + bb)^*$	It would be string of a's and b's of even length that can be obtained by concatenating any combination of the strings aa, ab, ba and bb including null i.e. $\{aa, ab, ba, bb, aaab, aaba, \dots\}$

Regular Sets & Their Properties

- It may be defined as the set that represents the value of the regular expression and consists specific properties.

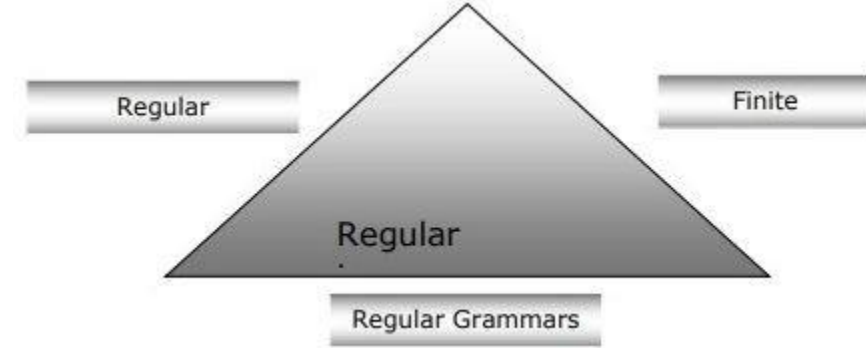
Properties of regular sets

- If we do the **union of two regular sets** then the resulting set would also **be regular**.
- If we do **the intersection of two regular** sets then the resulting set would also **be regular**.
- If we do the **complement of regular sets**, then the resulting set would also be **regular**.
- If we do **the difference of two regular sets**, then the resulting set would also be **regular**.
- If we do the **reversal of regular sets**, then the resulting set would also be **regular**.
- If we take the **closure of regular sets**, then the resulting set would also be **regular**.
- If we do the **concatenation of two regular sets**, then the resulting set would also be **regular**.

Finite State Automata

- The term automata, derived from **the Greek** word "**αὐτόματα**" meaning "**self-acting**", is the plural of automaton which may be defined as an abstract **self-propelled computing device** that follows a **predetermined sequence of operations automatically**.
- An **automaton having a finite number of states** is called a **Finite Automaton** (FA) or Finite State automata (FSA).
- Mathematically, an automaton can be represented by a **5-tuple $(Q, \Sigma, \delta, q_0, F)$** , where –
 - **Q** is a finite set of states.
 - **Σ** is a finite set of symbols, called the alphabet of the automaton.
 - **δ** is the transition function
 - **q_0** is the initial state from where any input is processed ($q_0 \in Q$).
 - **F** is a set of final state/states of Q ($F \subseteq Q$).

Relation between Finite Automata Regular Grammars and Regular



- As we know that finite state automata are the theoretical foundation of computational work and regular expressions is one way of describing them.
- We can say that any regular expression can be implemented as FSA and any FSA can be described with a regular expression.
- Regular expression is a way to characterize a kind of language called regular language.
- Regular language can be described with the help of both FSA and regular expression.
- Regular grammar, a formal grammar that can be right-regular or left-regular, is another way

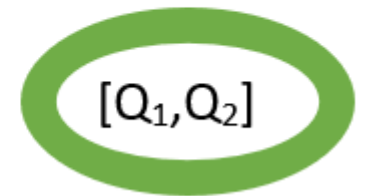
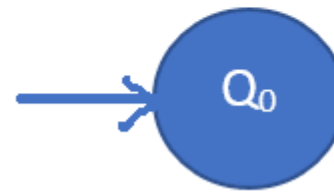
Types of Finite State Automation (FSA)

- **Deterministic Finite automation (DFA)**
- **Non-deterministic Finite Automation (NDFA)**

Types of Finite State Automation (FSA)

- **Deterministic Finite automation (DFA)**

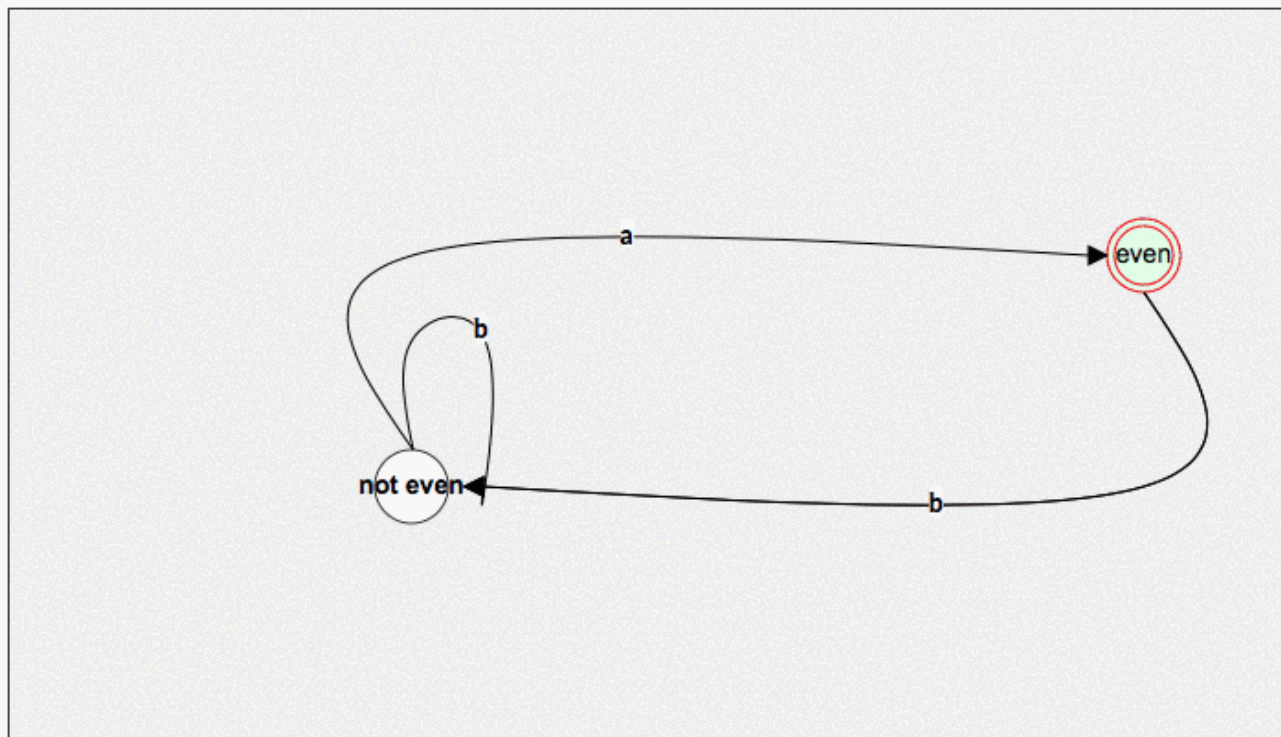
- It may be defined as the type of finite automation wherein, for **every input symbol** we can **determine** the **state to which** the **machine will move**.
- It has a **finite number of states** that is why the machine is called **Deterministic Finite Automaton (DFA)**.



- **Deterministic Finite automation (DFA)**

- Mathematically, a **DFA** can be represented by a 5-tuple **$(Q, \Sigma, \delta, q_0, F)$** , where –
- **Q** is a **finite set of states**.
- **Σ** is a **finite set of symbols**, called the **alphabet** of the automaton.
- **δ** is the **transition function** where $\delta: Q \times \Sigma \rightarrow Q$.
- **q_0** is the **initial state** from where any input is processed ($q_0 \in Q$).
- **F** is a set of **final state**/states of Q ($F \subseteq Q$).

DFA Application



ababab

Input:

+ Create diagram

▶ Run

⌚ Run by step

Alphabet:

Initial
state:

States

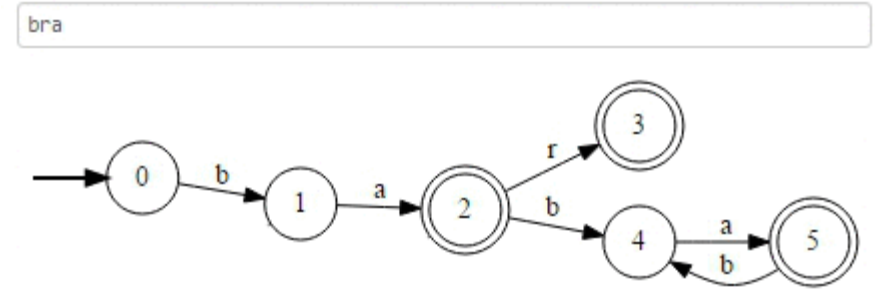
#	Description	Accepted	Delete
---	-------------	----------	--------

Transition Function

State	a	b
even	not even	not even

- **Deterministic Finite automation (DFA)**

- Whereas **graphically**, a DFA can be represented by **diagraphs** called **state diagrams** where –



- The **states** are represented by **vertices**.
- The **transitions** are shown by labeled **arcs**.
- The **initial state** is represented by an **empty incoming arc**.
- The **final state** is represented by **double circle**.

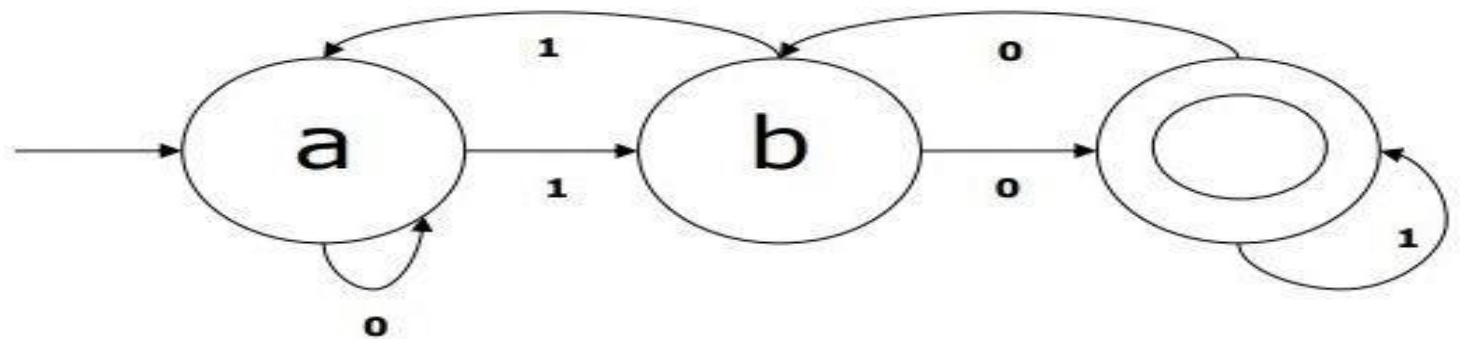
• Deterministic Finite automation (DFA)-Example

- Suppose a DFA be
- $Q = \{a, b, c\}$,
- $\Sigma = \{0, 1\}$,
- $q_0 = \{a\}$,
- $F = \{c\}$,

Transition function δ is shown in the table as follows

Current State	Next State for Input 0	Next State for Input 1
A	a	B
B	b	A
C	c	C

The graphical representation of this DFA would be as follows –



- **NON-Deterministic Finite automation (NDFA)**

- It may be defined as the type of **finite automation** where for every input symbol we cannot determine the state to which the machine will move i.e. the **machine can move to any combination of the states**. It has a **finite number of states** that is why the machine is called Non-deterministic Finite Automation (NDFA).

- **Non-Deterministic Finite automation (NDFA)**

- Mathematically, NDFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where
- Q is a finite set of states.
- Σ is a finite set of symbols, called the alphabet of the automaton.
- δ :-is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$.
- q_0 :-is the initial state from where any input is processed ($q_0 \in Q$).
- F :-is a set of final state/states of Q ($F \subseteq Q$).

- **Non-Deterministic Finite automation (NDFA)**

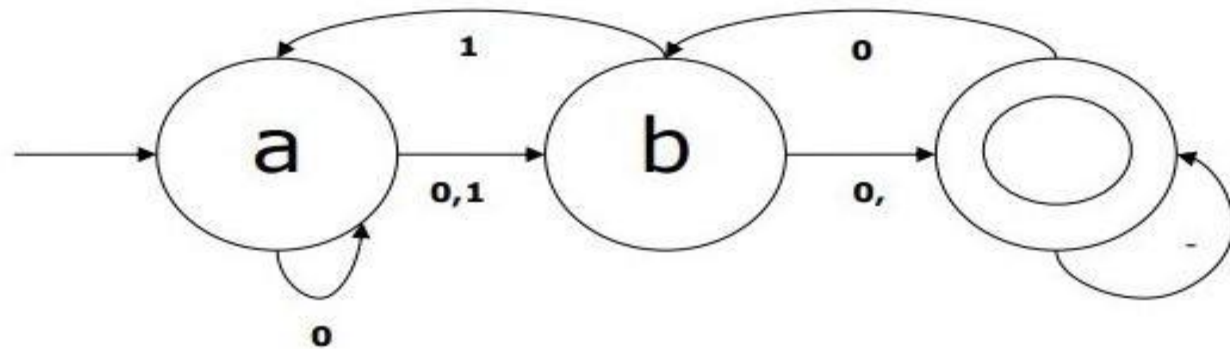
- Whereas graphically (same as DFA), a NDFA can be represented by **diagraphs** called **state diagrams** where –
- The states are represented by **vertices**.
- The transitions are shown by labeled **arcs**.
- The initial state is represented by an **empty incoming arc**.
- The final state is represented by double **circle**.

• Non-Deterministic Finite automation (NDFA)

- Suppose a NDFA be
- $Q = \{a, b, c\}$,
- $\Sigma = \{0, 1\}$,
- $q_0 = \{a\}$,
- $F = \{c\}$,
- Transition function δ is shown in the table as follows –

Current State	Next State for Input 0	Next State for Input 1
A	a, b	B
B	C	a, c
C	b, c	C

The graphical representation of this NDFA would be as follows –



• Morphological Parsing

- The term morphological parsing is related to the parsing of morphemes.
- We can define morphological parsing as the problem of recognizing that a word breaks down into smaller meaningful units called morphemes producing some sort of linguistic structure for it.
- For example, we can break the word *foxes* into two, *fox* and *-es*.
- We can see that the word *foxes*, is made up of two morphemes, one is *fox* and other is *-es*.

- **Morphological Parsing**

- In other sense, we can say that morphology is the study of –
- The **formation of words**.
- The **origin of the words**.
- **Grammatical forms of the words**.
- **Use of prefixes and suffixes** in the formation of words.
- How **parts-of-speech (PoS) of a language** are formed.

- **Morphological Parsing**

- Morphemes, the **smallest meaning-bearing units**, can be divided into two types –
 - **Stems**
 - **Word Order**

- **Stems**

- It is the core meaningful unit of word. We can also say that it is **the root of the word**. For example, in the **word foxes, the stem** is fox.
- **Affixes** – As the name suggests, they add some additional **meaning and grammatical functions to the words**. For example, in the **word foxes, the affix** is – es.
- Further, affixes can also be divided into following four types –
 - **Prefixes** – As the name suggests, prefixes precede the stem. For example, in the word **unbuckle, un** is the prefix.

• Stems

- **Suffixes** – As the name suggests, **suffixes follow the stem**. For example, in the word **cats**, **-s is the suffix**.
- **Infixes** – As the name suggests, **infixes are inserted inside the stem**. For example, the **word cupful**, **can be pluralized as cupsful** by using **-s** as the infix.
- **Circumfixes** – They precede **and follow the stem**. There are very less examples of circumfixes in English language. A very common example is **'A-ing'** where we can use **-A precede and -ing follows** the stem.

- **Word Order**

- The order of the words would be decided by **morphological parsing**. Let us now see the requirements for building a morphological parser –

- **Lexicon**

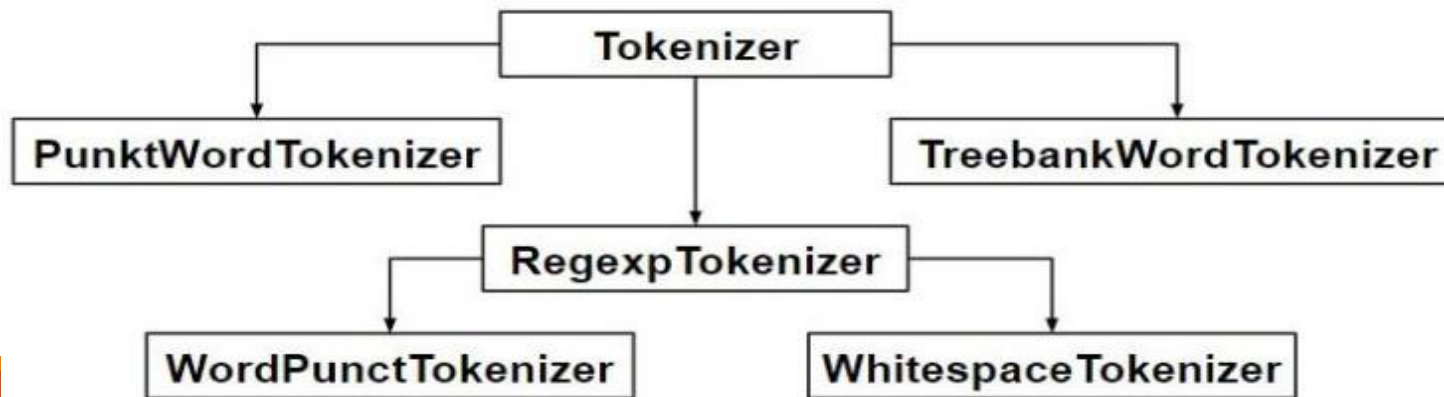
- The very first requirement for building a morphological parser is **lexicon**, which includes the **list of stems and affixes along with the basic information** about them. For example, the **information like whether the stem is Noun stem or Verb stem**, etc.

- **Morphotactic**
- It is basically the model of morpheme ordering. In other sense, the model **explaining which classes of morphemes** can follow other classes of morphemes inside a word. For example, the **morphotactic fact is that the English plural morpheme** always follows the noun rather than preceding it.
- **Orthographic rules**
- These **spelling rules are used to model the changes occurring in a word**. For example, the rule of converting **y to ie** in word like city+s
= cities not citys.

https://www.tutorialspoint.com/natural_language_processing/natural_language_processing_word_level_analysis.htm

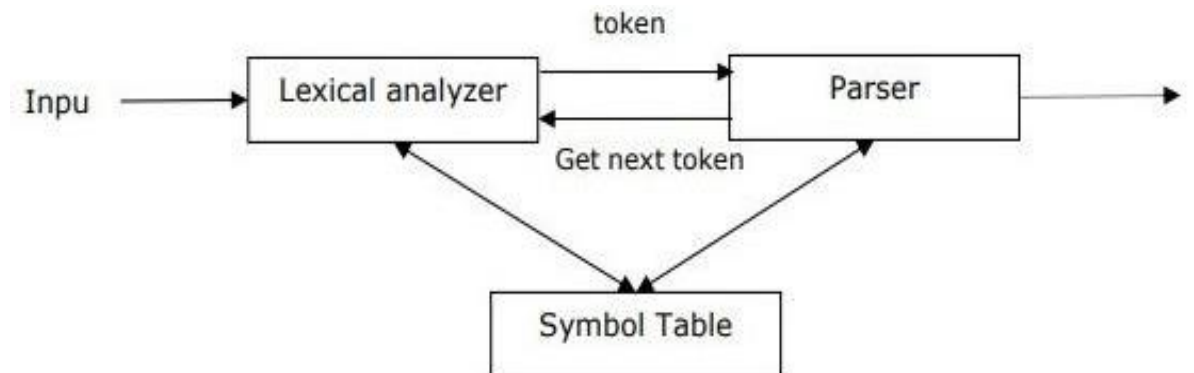
- **Word and Sentence Tokenization**

- Tokenization in simple words is **the process of splitting a phrase, sentence, paragraph, one or multiple text documents into smaller units.**
- Each of these smaller units is called **a token**. Now, these tokens can be anything — **a word, a subword, or even a character.**



- **Parser**

- It may be defined as the **software component designed for taking input data (text) and giving structural representation of the input after checking for correct syntax as per formal grammar.**
- It also builds a data structure generally in the form of **parse tree or abstract syntax tree or other hierarchical structure.**



Finite State Morphological Parsing

- The main roles of the parser include –
- To **report any syntax** error.
- To recover from **commonly occurring error so** that the processing of the remainder of program can be continued.
- To create parse tree.
- To create symbol table.
- To produce intermediate representations (IR).

Finite State Morphological Parsing

- Types of Parsing
- Derivation divides parsing into the followings two types –
- Top-down Parsing & Bottom-up Parsing
- **Top-down Parsing**: In this kind of parsing, the parser starts constructing the parse tree from the start symbol and then tries to transform the start symbol to the input.
- The most common form of topdown parsing uses recursive procedure to process the input. The main disadvantage of recursive descent parsing is backtracking.
- Bottom-up Parsing: In this kind of parsing, the parser starts with the input symbol and tries to construct the parser tree up to the start symbol.

Finite State Morphological Parsing

- Concept of Derivation: In order to **get the input string**, we need a **sequence of** production rules.
- Derivation is a set of production rules.
- During parsing, we need to decide the **non-terminal**, which is to be replaced along with deciding the **production rule with the help of which the non-terminal** will be replaced.

Finite State Morphological Parsing

- Types of Derivation
- **Left-most Derivation**
- In the left-most derivation, the **sentential form of an input is scanned and replaced from the left to the right.**
- The sentential form in this case is called the left-sentential form.
- **Right-most Derivation:**
- In the left-most derivation, the sentential form of **an input is scanned and replaced from right to left.**
- The sentential form in this case is called the right-sentential form.

Finite State Morphological Parsing

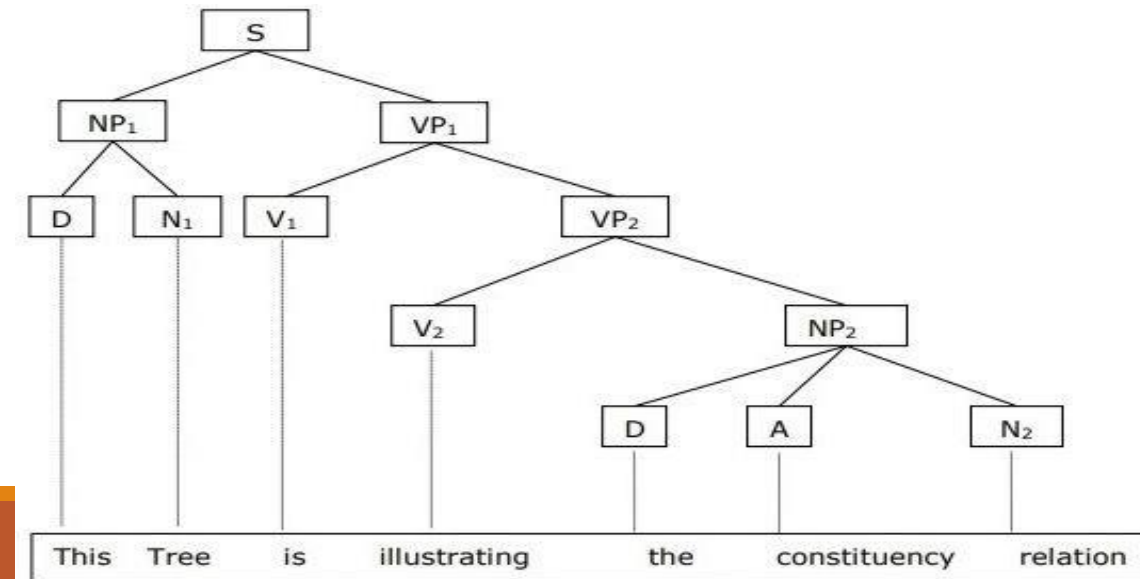
- **Concept of Grammar**
- Grammar is very essential and important to describe the syntactic structure of well-formed programs.
- In the literary sense, they denote syntactical rules for conversation in natural languages.
- Linguistics have attempted to define grammars since the inception of natural languages like **English, Hindi, etc.**
- The theory of **formal languages is also applicable in the fields of Computer Science mainly in programming languages and data structure.**
- For example, **in 'C' language, the precise grammar rules** state how functions are made from lists and statements.

Finite State Morphological Parsing

- Mathematically, a grammar G can be formally written as a 4-tuple (N, T, S, P) where –
- N or VN = set of non-terminal symbols, i.e., variables. or Σ = set of terminal symbols.
- S = Start symbol where $S \in N$
- P denotes the Production rules for Terminals as well as Non-terminals.
- It has the form $\alpha \rightarrow \beta$, where α and β are strings on $VN \cup \Sigma$ and least one symbol of α belongs to VN

Finite State Morphological Parsing

- Before giving an example of constituency grammar, we need to know the fundamental points about constituency grammar and constituency relation.
- All the related frameworks view the sentence structure in terms of constituency relation.
- The constituency relation is derived from the subject-predicate division of Latin as well as Greek grammar.
- The basic clause structure is understood in terms of a noun phrase NP and a verb phrase VP.
- We can write the sentence
- **“This tree is illustrating the constituency relation”**



Finite State Morphological Parsing

Example

Before giving an example of Dependency grammar, we need to know the fundamental points about Dependency grammar and Dependency relation.

In DG, the linguistic units, i.e., words are connected to each other by directed links. The verb becomes the centre of the clause structure.

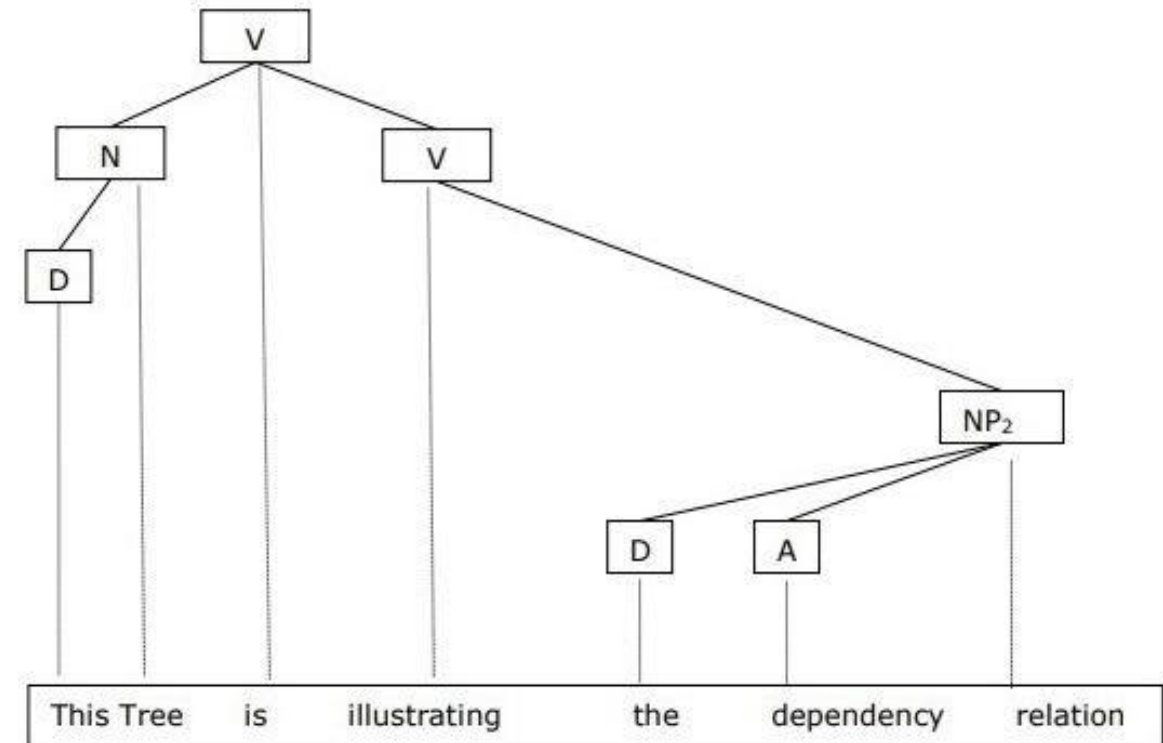
Every other syntactic units are connected to the verb in terms of directed link.

These syntactic units are called dependencies.

We can write the sentence

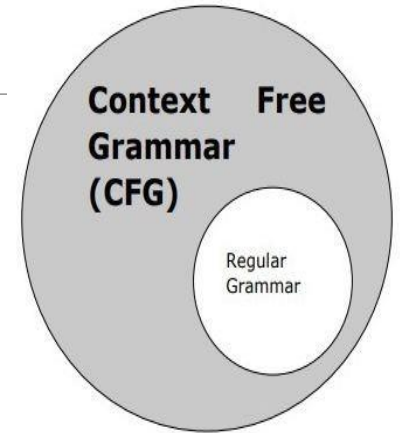
“This tree is illustrating th dependency relation”

as follows;



Context free grammar

- **Context free grammar, also called CFG, is a notation for describing languages and a superset of Regular grammar.**



- CFG consists of finite set of grammar rules with the following four components –

- **Set of Non-terminals**-It is denoted by V .
- The non-terminals are syntactic variables that denote the sets of strings, which further help defining the language, generated by the grammar.

Set of Terminals -It is also called tokens and defined by Σ . Strings are formed with the basic symbols of terminals.

Context Free Grammar

Set of Productions

- It is denoted by P . The set defines how the terminals and non-terminals can be combined. Every production(P) consists of non-terminals, an arrow, and terminals (the sequence of terminals).
- Non-terminals are called the left side of the production and terminals are called the right side of the production.

Context Free Grammar

Start Symbol

The **production begins from the start symbol**. It is denoted by symbol S .

Non-terminal symbol is always designated as start symbol.

Context Free Grammar

Studying meaning of individual word

It is **the first part of the semantic analysis** in which the study of the meaning of **individual words is performed**. This part is **called lexical semantics**.

Studying the combination of individual words

In the second part, the individual words will be combined to provide meaning in sentences.

Porter stemmer

The Porter stemming algorithm is a process for removing suffixes from words in English. Removing suffixes automatically is an operation which is especially useful in the field of information retrieval. In a typical IR environment a document is represented by a vector of words, or terms. Terms with a common stem will usually have similar meanings, for example:

CONNECT
CONNECTED
CONNECTING
CONNECTION
CONNECTIONS

Frequently, the performance of an IR system will be improved if term groups such as this are conflated into a single term. This may be done by removal of the various suffixes -ED, -ING, -ION, IONS to leave the single term CONNECT. In addition, the suffix stripping process will reduce the total number of terms in the IR system, and hence reduce the size and complexity of the data in the system, which is always advantageous.

Suffix Stripping

- It is an operation by which suffixes are removed from terms.
- A document is represented by a vector of words, or terms.
- Terms with a common stem:
 - Connect
 - Connected
 - Connecting
 - Connection
 - Connections



Connect
(Root word)

Stemming

- In linguistic morphology and information retrieval, **stemming** is the process of reducing terms to their "roots" before indexing.
- Stemming programs are commonly referred to as stemming algorithms or stemmers.
- "Stemming" suggest crude affix chopping
 - language dependent
 - e.g., **automate(s), automatic, automation** all reduced to **automat**.

Porter Stemmer

- Common algorithm for stemming English
- The algorithm consists of 5 sets of rules, applied in order
 - phases applied sequentially
 - each phase consists of a set of commands
 - **Sample convention:** Of the rules in a compound command, select the one that applies to the longest suffix.

Measure of Word

- $M=0$ TREE, BY, TR
- $M=1$ TROUBLE, OATS, TREES, IVY
- $M=2$ TROUBLES, PRIVATE, OATEN, ORRERY

Rule for removing a suffix

- The rules are of the form:

(condition) S1 -> S2

Where S1 and S2 are suffixes.

This means that if a word ends with a suffix S1, and the stem before S1 satisfies the given condition then S1 is replaced by S2.

- E.g. if rule is (m>1) EMENT ->
 - In this S1 is EMENT and S2 is NULL
 - So, this would map REPLACEMENT to REPLAC

Rule for removing a suffix

- The condition part may also contain the following:

m m	The measure of the stem
*S	The stem ends with S
v	The stem contains a vowel
*d	The stem ends with a double consonant (TT, SS)
*o	The stem ends in CVC (second C not W, X, or Y) E.g. WIL, HOP

- The condition part may also contain expressions with and, or and not.
 - E.g. (m>1 and (*s or *t)) : tests for a stem with m>1 ending in s or t

The Porter Stemmer: Step 1a

- **SSES -> SS**
 - *caresses -> caress*
- **IES -> I**
 - *ponies -> poni*
 - *ties -> ti*
- **SS -> SS**
 - *caress -> caress*
- **S -> ε**
 - *cats -> cat*

The Porter Stemmer: Step 1b

- **(m>1) EED -> EE**
 - Condition verified: *agreed* -> *agree*
 - Condition not verified: *feed* -> *feed*
- **(*V*) ED -> €**
 - Condition verified: *plastered* -> *plaster*
 - Condition not verified: *bled* -> *bled*
- **(*V*) ING -> €**
 - Condition verified: *motoring* -> *motor*
 - Condition not verified: *sing* -> *sing*

The Porter Stemmer: Step 1c and 2

- Step 1c: Y Elimination (*V*) Y -> I
 - Condition verified: happy -> happi
 - Condition not verified: sky -> sky
- Step 2: Derivational Morphology, I
 - (m>0) ATIONAL -> ATE
 - Relational -> relate
 - (m>0) IZATION -> IZE
 - generalization -> generalize
 - (m>0) BILITI -> BLE
 - sensibiliti -> sensible

The Porter Stemmer: Steps 3 and 4

- Step 3: Derivational Morphology, II

- (m>0) ICATE -> IC
 - *triplicate* -> *triplic*
- (m>0) FUL -> ϵ
 - *hopeful* -> *hope*
- (m>0) NESS -> ϵ
 - *goodness* -> *good*

- Step 4: Derivational Morphology, III

- (m>0) ANCE -> ϵ
 - *allowance* -> *allow*
- (m>0) ENT -> ϵ
 - *dependent* -> *depend*
- (m>0) IVE -> ϵ
 - *effective* -> *effect*

The Porter Stemmer: Step 5 (cleanup)

- Step 5a

- (m>1) E -> ε
 - *probate* -> *probat*
- (m=1 & !*o) NESS -> ε
 - *goodness* -> *good*

- Step 5b

- (m>1 & *d & *L) -> single letter
 - Condition verified: *control* -> *control*
 - Condition not verified: *roll* -> *roll*

1. What is spelling correction in NLP?

Spelling correction in NLP refers to detecting incorrect spellings and then correcting them.

2. How does autocorrect work in NLP?

Autocorrect tries first to find whether a given word is correct or not. It does so by checking in the dictionary. If the word exists, it means that it is correct; otherwise, it is not. If the word isn't right, it tries to find other close options and finds the best-suited word.

3. How do you correct spelling in Python?

We can correct spellings in Python using SymSpell, Norvig's method, etc.

4. Is autocorrect artificial intelligence?

Yes, auto correction is artificial intelligence as it uses the intelligence of machines to correct spelling.

5. Is spelling important in Python?

Yes, spelling is very important in Python as we can get correct and accurate results with correct spellings. With incorrect spelling, the accuracy of tasks like search engines can decrease due to their ambiguous nature.



Real-word spelling errors

- ...leaving in about fifteen minuets to go to her house.
 - The design an construction of the system.
 - Can they lave him my messages?
 - The study was conducted mainly be John Black.
-
- 25-40% of spelling errors are real words Kukich 1992



Solving real-world spelling errors

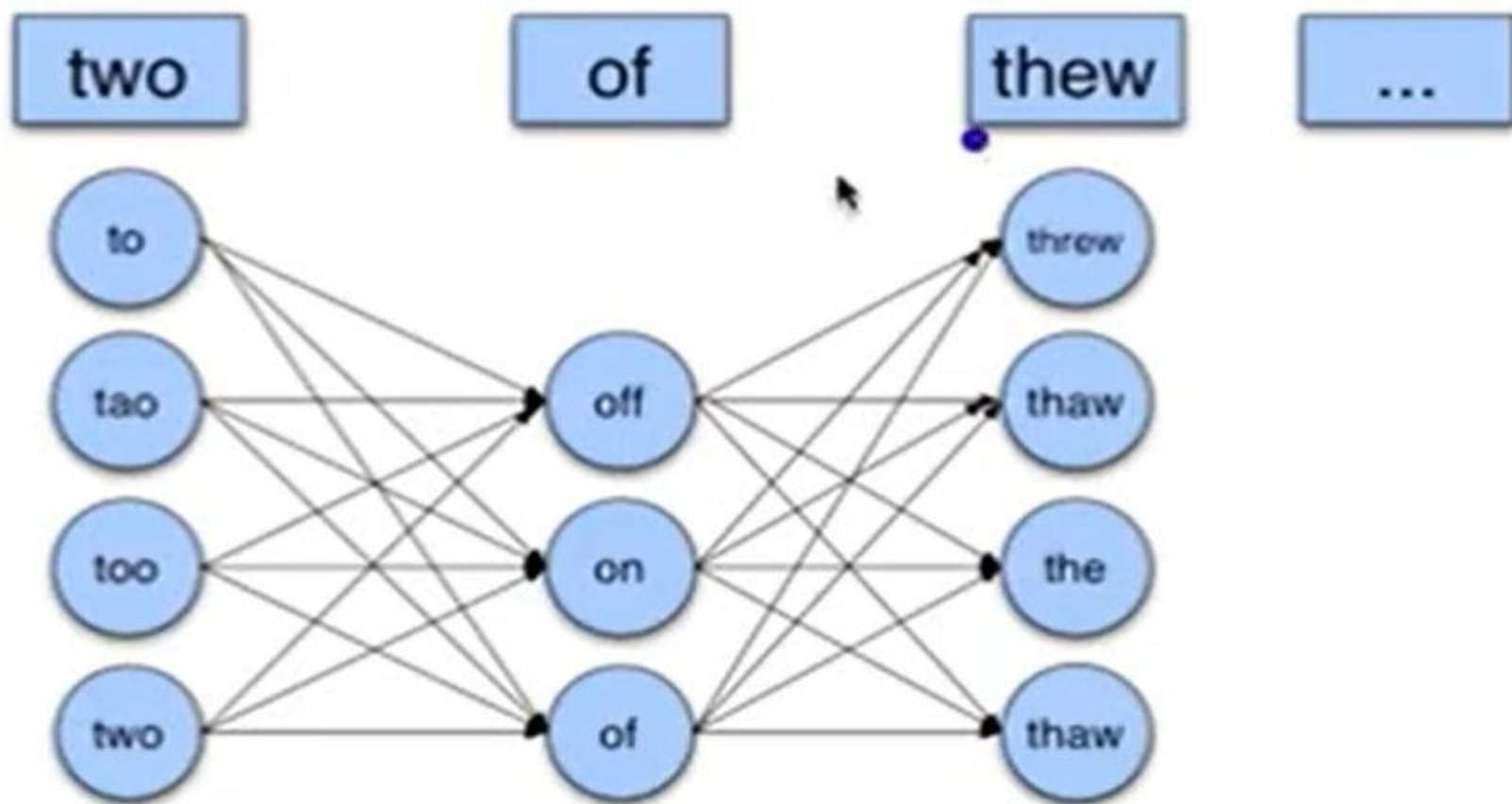
- For each word in sentence
 - Generate *candidate set*
 - the word itself
 - all single-letter edits that are English words
 - words that are homophones
- Choose best candidates
 - Noisy channel model
 - Task-specific classifier



Noisy channel for real-word spell correction

- Given a sentence $w_1, w_2, w_3, \dots, w_n$
- Generate a set of candidates for each word w_i
 - $\text{Candidate}(w_1) = \{w_1, w'_1, w''_1, w'''_1, \dots\}$
 - $\text{Candidate}(w_2) = \{w_2, w'_2, w''_2, w'''_2, \dots\}$
 - $\text{Candidate}(w_n) = \{w_n, w'_n, w''_n, w'''_n, \dots\}$

Noisy channel for real-word spell correction





Simplification: One error per sentence

- Out of all possible sentences with one word replaced
 - w_1, w''_2, w_3, w_4 two off thew
 - w_1, w_2, w'_3, w_4 two of the
 - w'''_1, w_2, w_3, w_4 too of thew
 - ...
- Choose the sequence W that maximizes $P(W)$



Where to get the probabilities

- Language model
 - Unigram
 - Bigram
 - Etc
- Channel model
 - Same as for non-word spelling correction
 - Plus need probability for no error, $P(w|w)$



Probability of no error

- What is the channel probability for a correctly typed word?
- $P(\text{"the"} | \text{"the"})$
- Obviously this depends on the application
 - .90 (1 error in 10 words)
 - .95 (1 error in 20 words)
 - .99 (1 error in 100 words)
 - .995 (1 error in 200 words)



Peter Norvig's "thew" example

x	w	x w	$P(x w)$	$P(w)$	$10^9 P(x w)P(w)$
thew	the	ew e	0.0000007	0.02	144
thew	thew		0.95	0.000000009	90
thew	thaw	e a	0.001	0.00000007	0.7
thew	threw	h hr	0.0000008	0.0000004	0.03
thew	thwe	ew we	0.0000003	0.000000004	0.0001

Minimum Edit Distance

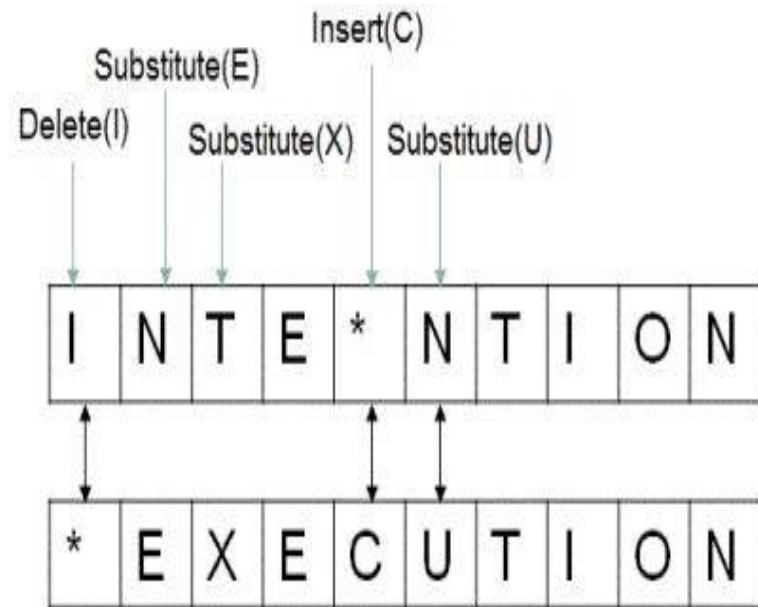
Minimum Edit distance between two strings str1 and str2 is defined as **the minimum number of insert/delete/substitute operations required to transform str1 into str2.**

For example if **str1 = "ab"**, **str2 = "abc"** then making **an insert operation** of character 'c' on str1 transforms str1 into str2. also **calculate edit distance** as number of operations required to transform str2 into str1.

For above example, if we perform a **delete operation** of character 'c' on str2, it is transformed into str1 resulting in same edit distance of 1.

Example, if str1 = "INTENTION" and str2 = "EXECUTION", then the minimum edit distance between str1 and str2 turns out to be 5 as shown below.

All operations are performed on str1



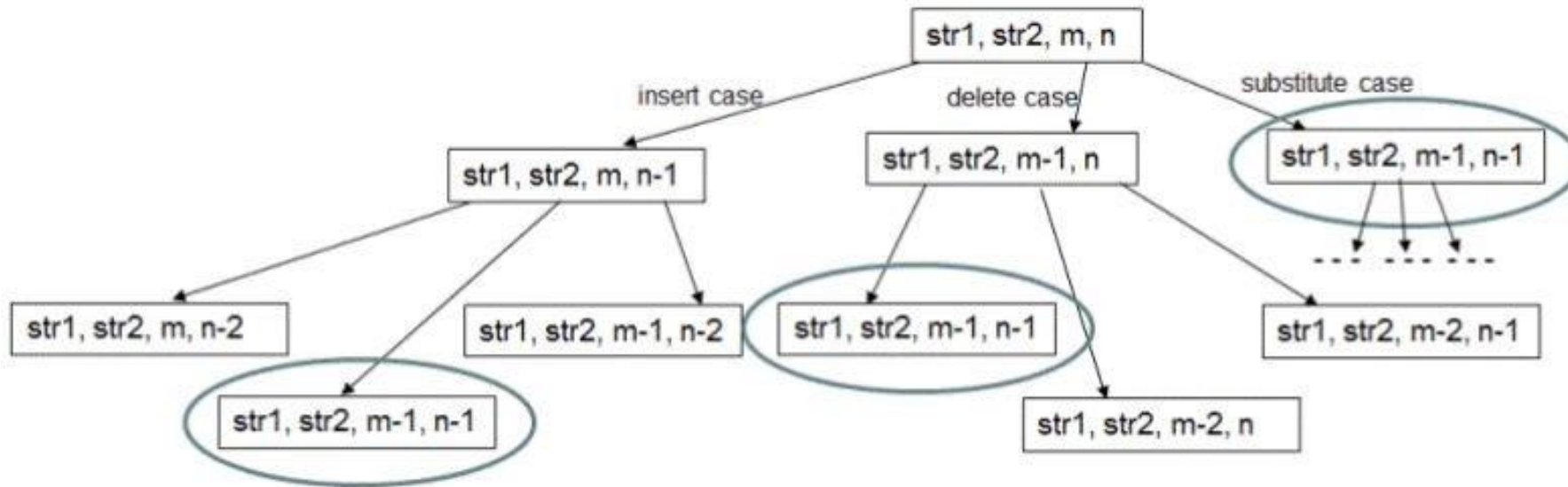
```
public int findEditDistance(String str1, String str2, int m, int n)
{
    if (str1.charAt(m) == str2.charAt(n))
    {
        return findEditDistance(str1, str2, m-1, n-1);
    }
    return min(
        1 + findEditDistance(str1, str2, m-1, n),
        1 + findEditDistance(str1, str2, m, n-1),
        1 + findEditDistance(str1, str2, m-1, n-1)
    );
}
```

1.If length of str1 is 0, then we need to insert all characters of str2 into str1 to make them equal. Therefore, edit distance between str1 and str2 when str1 is empty is length of str2.

2.If length of str2 is 0, then we need to delete all characters of str1 to make it equal to str2. Therefore, edit distance between str1 and str2 when str2 is empty is length of str1.

Dynamic Programming approach: Now let's see how we can optimize the time complexity of this algorithm.

The partial recursion tree of call sequence for function `findDistance(String str1, String str2, int m, int n)` would look like



As highlighted above, there are function calls with same arguments which are being computed **again and again**. To avoid this **redundancy** we use **Dynamic Programming Approach**.

- | - | O | E | X | E | C | U | T | I | O | N |
|---|---|---|---|---|---|---|---|---|---|---|
| O | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| I | 1 | 1 | 2 | 3 | 4 | 5 | 6 | 6 | 7 | 8 |
| N | 2 | 2 | 2 | 3 | 4 | 5 | 6 | 7 | 7 | 7 |
| T | 3 | 3 | 3 | 3 | 4 | 5 | 5 | 6 | 7 | 8 |
| E | 4 | 3 | 4 | 3 | 4 | 5 | 6 | 6 | 7 | 8 |
| N | 5 | 4 | 4 | 4 | 4 | 5 | 6 | 7 | 7 | 7 |
| T | 6 | 5 | 5 | 5 | 5 | 5 | 5 | 6 | 7 | 8 |
| I | 7 | 6 | 6 | 6 | 6 | 6 | 6 | 5 | 6 | 7 |
| O | 8 | 7 | 7 | 7 | 7 | 7 | 7 | 6 | 5 | 6 |
| N | 9 | 8 | 8 | 8 | 8 | 8 | 8 | 7 | 6 | 5 |

For this computation, we simply have to do - $(1 + \text{array}[m-1][n])$ where 1 is the cost of delete operation and $\text{array}[m-1][n]$ is edit distance between 'm-1' characters of str1 and 'n' characters of str2.

2. Similarly, for the second case of inserting last character of str2 into str1, we have to do - (1 + array[m][n-1]).

```
public class EditDistanceStrings
{
    final static int ERROR_INPUT = -1;

    private int min(int a, int b)
    {
        return (a<b)?a:b;
    }

    private int min(int a, int b, int c)
    {
        return min(min(a,b),c);
    }
}
```

```
public int findDistance(String str1, String str2, int m, int n)
{
    // if either of the strings is null, distance cannot be
    computed
    if (str1 == null || str2 == null)
    {
        return ERROR_INPUT;
    }

    // if str1 is an empty string, then insert all 'n' characters
    of str2 into str1
    if (m == 0)
    {
        return n;
    }

    // if str2 is an empty string, then remove all 'm' characters
    of str1
    if (n == 0)
    {
        return m;
    }
}
```

```

/*
 * if last characters of str1 and str2 are equal
 * then we need to calculate distance for strings ending at
second last index
 * for both str1 and str2
 */
if (str1.charAt(m-1) == str2.charAt(n-1))
{
    return findDistance(str1, str2, m-1, n-1);
}
/*
 * return minimum of following three cases:
 * delete last character of str1 and check distance: 1 + distance(str1, str2,
m-1, n)
 * insert last character of str2 into str1 and check distance: 1 + distance(str1,
str2, m, n-1)
 * replace last char of str1 with last char of str2 and check distance: 1 +
distance(str1, str2, m-1, n-1)
 */
return min (
    1 + findDistance(str1, str2, m-1, n),
    1 + findDistance(str1, str2, m, n-1),
    1 + findDistance(str1, str2, m-1, n-1)
);
}

```

```

public int findDistance(String str1, String str2)
{

```

```

    // if either of the strings is null, distance cannot be computed

```

```
// all values are by default initialized to 0 by JVM
int[][] distanceTable = new
int[str1.length()+1][str2.length()+1];

int numRows = str1.length() + 1;
int numCols = str2.length() + 1;

for (int m = 0; m < numRows; m++)
{
    for (int n = 0; n < numCols; n++)
    {
        // if length of str1 is 0, we have no option but to insert
all of str2
        if (m == 0)
        {
            distanceTable[m][n] = n;
        }

        // if length of str2 is 0, delete all of str1 to make it
match with str2
        else if (n == 0)
        {
            distanceTable[m][n] = m;
        }
    }
}
```

```

*/
    else if (str1.charAt(m-1) == str2.charAt(n-1))
    {
        distanceTable[m][n] = distanceTable[m-1][n-1];
    }

    /*
    * else use minimum of following three cases:
    * delete last character of str1 and check distance:
distance(str1, str2, m-1, n)
    * insert last character of str2 into str1 and check
distance: distance(str1, str2, m, n-1)
    * replace last char of str1 with last char of str2 and
check distance: distance(str1, str2, m-1, n-1)
    */
    else
    {
        distanceTable[m][n] = min (
            1 + distanceTable[m-1][n],
            1 + distanceTable[m][n-1],
            1 + distanceTable[m-1][n-1]
        );
    }
}
}
}

```

```
return distanceTable[numRows-1][numCols-1];  
}
```

```
public static void main(String[] args)  
{  
    EditDistanceStrings solution = new EditDistanceStrings();  
  
    System.out.print("minimum edit distance between \"intention\"  
and \"execution\" is: \n");  
    System.out.println(solution.findDistance("intention",  
"execution"));  
}
```


Human morphological processing:

- *Natural Language Processing (NLP) is one of the most rapidly growing areas of research.*

- *Morphological Analysis and Morphological Generation might be considered highly relevant in **most Natural Language Processing applications**.*

- Because morphological analysis is a technique for recognizing a word, the result can be employed at a later stage.

- such as **spell checkers** and **machine translation**.

- Morphological analysis is a field of linguistics that studies the structure of words.

- It identifies how a word is produced through the use of morphemes.

A morpheme is a basic unit of the English language.

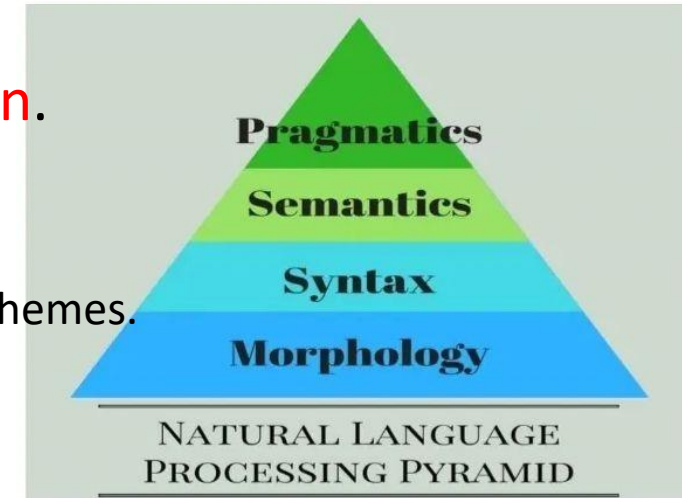
- The morpheme is the smallest element of a word that has grammatical function and meaning.

- Free morpheme and bound morpheme are the two types of morphemes.

- A **single free morpheme** can become a complete word.

For instance, **a bus, a bicycle**, and so forth.

A **bound morpheme**, on the other hand, cannot stand alone and must be joined to a free morpheme to produce a word. **ing, un, and other bound** morphemes are examples.



What is a morphological analyzer ?

- words are formed through morphological processes such as **affixation**.
- For example, by adding the **suffix '-s'** to the verb **'to dance'**, we form the third person singular **'dances'**.
- A morphological analyzer assigns the attributes of a given word by evaluating what morphological processes the form has undergone.
- If you give it the word **'bailaré' in Spanish**, it will tell you it is the **first person**, singular, simple future, indicative form of the **verb 'bailar'**.

Morphological Parsing.

- It is the process of determining the **morphemes** from which a given word is constructed.
- Morphemes are the **smallest meaningful words** which cannot be divided further.
- Morphemes can be **stem or afix**.
- Stem are **the root** word whereas afix can be **prefix, suffix or infix**.
- For example-
Unsuccessfull → **un** success **ful**
(prefix) (stem) (suffix)
- Order of words also decide the morphological parser.
- To design a morphological parser we require **lexicon, morphptactics and orthographic rules**.

Types of Morphology:

- **Inflectional Morphology**

Inflectional morphology is the study of processes, including affixation and vowel change, that distinguish word forms in certain grammatical categories.

Inflectional morphology consists of at least five categories, provided in the following excerpt from Language Typology and Syntactic Description: Grammatical Categories and the Lexicon.

As the text will explain, derivational morphology cannot be so easily categorized because **derivation isn't as predictable as inflection. Examples- cats, men etc.**

- **Derivational Morphology:**

Is defined as morphology that creates new lexemes, either by changing the syntactic category (part of speech) of a base or by adding substantial, non grammatical meaning or both.

On the one hand, derivation may be distinguished from inflectional morphology, which typically does not change category but rather modifies lexemes to fit into various syntactic contexts;

inflection typically expresses distinctions like **number, case, tense, aspect, person**, among others.

On the other hand, derivation may be distinguished from compounding, which also creates new lexemes, but by combining two or more bases rather than by **affixation, reduplication, subtraction, or internal modification of various sorts.**

Distinguish is not so easy

Example of **Inflectional Morphology**

French

1st	je chante	<i>I sing</i>
2d	tu chantes	<i>you sing</i>
3d	elle chante	<i>she sings</i>
1st	nous chantons	<i>we sing</i>
2d	vous chantez	<i>you (pl.) sing</i>
3d	elles chantent	<i>they sing</i>

two sentences in German: The first one, *Der Junge sieht Sofia*, means that, “The boy sees Sofia”.

Look at the form of the phrase, *the boy*, “*der Junge*”.

Now, look at this other sentence, *Sofia sieht den Jungen*, which means that “Sofia sees the boy”.

In the first sentence, the boy is doing the seeing, but in the second, the boy is getting seen, and the word for boy, *Junge* has a different morpheme on it to indicate its different role in the sentence.

If we start with an adjective like *happy* and add the suffix *-ness*, we derive the noun that refers to the state of being that adjective, *happiness*.

Adding the suffix *-ful* to a noun derives an adjective, like *hopeful*.

Adding the suffix *-ize* to an adjective like *final* derives a verb like *finalize*.

Verb	Suffix	Noun
teach	-er	teacher
sing	-er	singer
dance	-er	dancer
bake	-er	baker
write	-er	writer

Noun	Suffix	Adjective
hope	-ful	hopeful
joy	-ful	joyful
care	-ful	careful
dread	-ful	dreadful

Adjective	Suffix	Verb
final	-ize	finalize
modern	-ize	modernize
social	-ize	socialize
public	-ize	publicize

Each of the morphologically complex derived words is related in meaning to the base, but it has a new meaning of its own.

English also derives new words by prefixing, and while adding a derivational prefix does lead to a new word with a new meaning, it often doesn't lead to a category change.

Prefix	Verb	Verb
re-	write	rewrite
re-	read	reread
re-	examine	reexamine
re-	assess	reassess

- **N-Grams: Building N-gram for spelling corrections, N-gram for language model.**

Uni-Gram	This	Is	Big	Data	AI	Book
Bi-Gram	This is	Is Big	Big Data	Data AI	AI Book	
Tri-Gram	This is Big	Is Big Data	Big Data AI	Data AI Book		

Given a sequence of N-1 words, an N-gram model predicts the most probable word that might follow this sequence. It's a probabilistic model that's trained on a corpus of text.

Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input.

An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities.

Since a simple N-gram model has limitations, improvements are often made via smoothing, interpolation and backoff.

An N-gram model is one type of a **Language Model (LM)**, which is about finding the probability distribution over word sequences.

Consider two sentences:

"There was heavy rain" vs. "There was heavy flood".

we know that the former sentence sounds better.

An N-gram model will tell us that "**heavy rain**" occurs much more often than "**heavy flood**" in the training corpus. Thus, the first sentence is more probable and will be selected by the model.

A model that simply relies on how often a word occurs without looking at previous words is called **unigram**.

If a model considers only the previous word to predict the current word, then it's called **bigram**.

If two previous words are considered, then it's a **trigram** model.

An n-gram model for the above example would calculate the following probability:

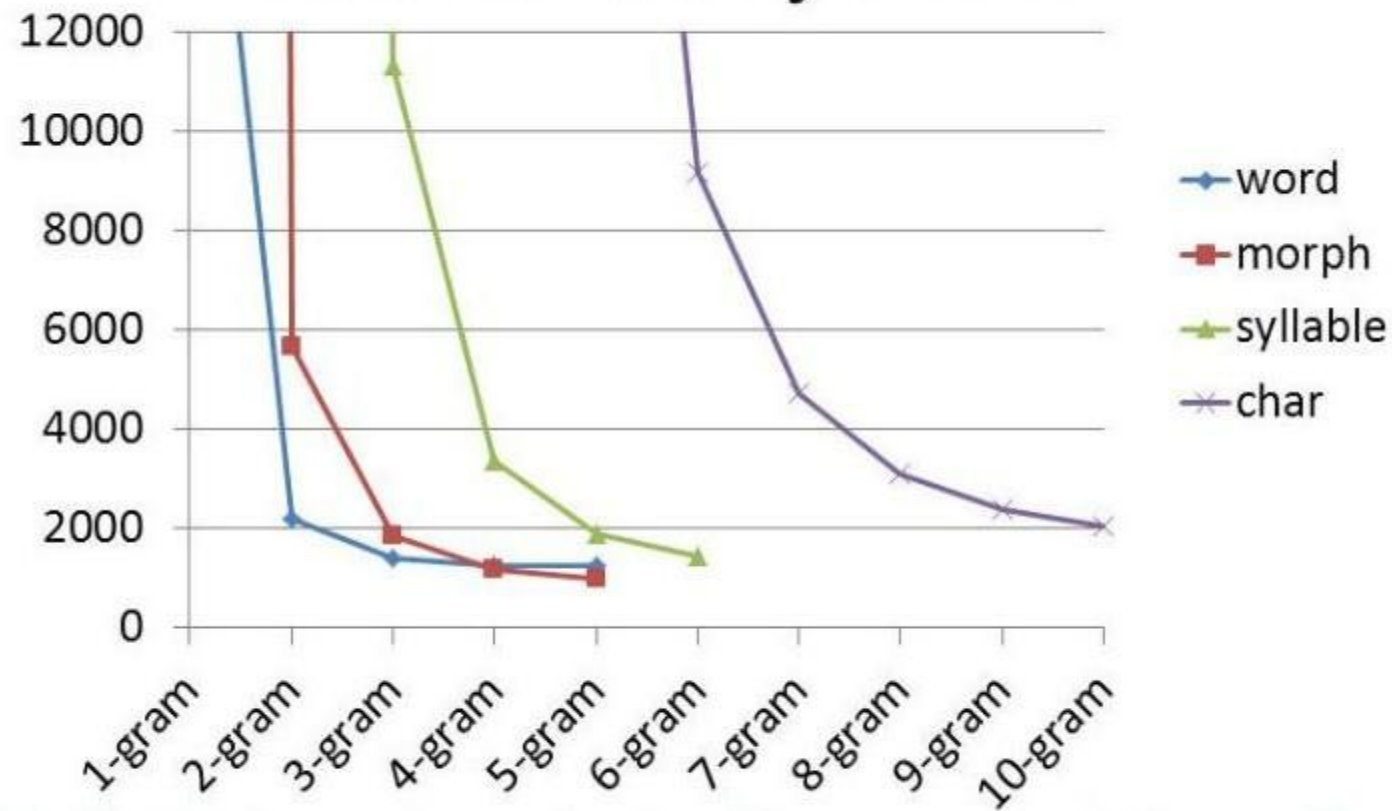
$$P(\text{'There was heavy rain'}) = P(\text{'There'}, \text{'was'}, \text{'heavy'}, \text{'rain'}) = \\ P(\text{'There'})P(\text{'was'}|\text{'There'})P(\text{'heavy'}|\text{'There was'})P(\text{'rain'}|\text{'There was heavy'})$$

Since it's impractical to calculate these conditional probabilities, using *Markov assumption*, we approximate this to a bigram model:

$$P(\text{'There was heavy rain'}) \sim P(\text{'There'})P(\text{'was'}|\text{'There'})P(\text{'heavy'}|\text{'was'})P(\text{'rain'}|\text{'heavy'})$$

- In speech recognition, **input may be noisy** and this can lead to wrong speech-to-text conversions.
- N-gram models can correct this based on their knowledge of the probabilities.
- Likewise, **N-gram models are used in machine translation** to produce more natural sentences in the target language.
- When correcting for spelling errors, sometimes dictionary lookups will not help.
- For example, in the phrase **"in about fifteen minuets"** the word 'minuets' is a valid dictionary word but it's incorrect in this context. N-gram models can correct such errors.
- N-gram models are usually at word level. It's also been used at character level to do stemming, that is, separate the root word from the suffix.
- By looking at N-gram statistics, we could also classify languages or differentiate between US and UK spellings.
- For example, 'sz' is common in Czech; 'gb' and 'kp' are common in Igbo.
- In general, many NLP applications benefit from N-gram models including **part-of-speech** tagging, natural language generation, word similarity, sentiment extraction and predictive text input.

Perplexity comparison of various n-grams normalized by words



- Perplexities by various unit sets will converge to similar results.
- Slight gain by longer units with smaller size of n.
- Morpheme slightly outperformed word, because of small OOV rate.

- The best way to evaluate a model is to check how well it predicts in end-to-end application testing.
- This approach is called **extrinsic evaluation** but it's time consuming and expensive.
- An alternative approach is to define a suitable metric and evaluate independent of the application. This is called **intrinsic evaluation**.
- This doesn't guarantee application performance but it's a quick first step to check algorithmic performance.
- A common metric is to use **perplexity**, often written as PP. Given a test set Because of the inverse relationship with probability, minimizing perplexity implies maximizing the test set probability.
- **Perplexity can also be related to the concept of entropy in information theory.**
- It's important in any **N-gram model to include markers** at start and end of sentences.
- This ensures that the total probability of the whole language sums to one. However, all calculations should include the end markers but not the start markers in the count of word tokens.

What are some limitations of N-gram models?

- A model trained on the works of Shakespeare will not give good predictions when applied to another genre. We need to therefore ensure that the training **corpus looks similar to the test corpus**.
- There's also the problem of **Out of Vocabulary (OOV) words**. These are words that appear **during testing but not in training**. One way to solve this is to start with a fixed vocabulary and convert **OOV words in training to UNK pseudo-word**.
- when applied to sentiment analysis, **a bigram model outperformed a unigram model but the number of features doubled**. Thus, scaling N-gram models to larger datasets or moving to a higher N needs good feature selection techniques.
- **N-gram models poorly capture longer-distance context**. It's been shown that after **6-grams, performance gains are limited**. Other language models such as Language modeling (LM) topic-based LM and latent semantic indexing do better.

What software tools are available to do N-gram modelling?

R has a few useful packages including *ngram*, *tm*, *tau* and *RWeka*. Package *tidytext* has functions to do N-gram analysis.

In Python, NLTK has the function `nltk.utils.ngrams()`. A more comprehensive package is [nltk.lm](#). Outside NLTK, the *ngram* package can compute n-gram string similarity.

```
//n-gram example
# imports
import string
import random
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('reuters')
from nltk.corpus import reuters
from nltk import FreqDist
# input the reuters sentences
sents = reuters.sents()

# write the removal characters such as : Stopwords and punctuation
stop_words = set(stopwords.words('english'))
string.punctuation = string.punctuation + "'" + '"' + '-' + ' ' + ' ' + ' ' + '-'
string.punctuation
removal_list = list(stop_words) + list(string.punctuation) +
['lt', 'rt']
removal_list
```

```
# generate unigrams bigrams trigrams
unigram=[]
bigram=[]
trigram=[]
tokenized_text=[]
for sentence in sents:
    sentence = list(map(lambda x:x.lower(),sentence))
    for word in sentence:
        if word== '.':
            sentence.remove(word)
        else:
            unigram.append(word)

    tokenized_text.append(sentence)
    bigram.extend(list(ngrams(sentence, 2,pad_left=True,
pad_right=True)))
    trigram.extend(list(ngrams(sentence, 3, pad_left=True,
pad_right=True)))
```

```
# remove the n-grams with removable words
```

```
def remove_stopwords(x):  
    y = []  
    for pair in x:  
        count = 0  
        for word in pair:  
            if word in removal_list:  
                count = count or 0  
            else:  
                count = count or 1  
        if (count==1):  
            y.append(pair)  
    return (y)  
unigram = remove_stopwords(unigram)  
bigram = remove_stopwords(bigram)  
trigram = remove_stopwords(trigram)
```



```

# generate frequency of n-grams
freq_bi = FreqDist(bigram)
freq_tri = FreqDist(trigram)

d = defaultdict(Counter)
for a, b, c in freq_tri:
    if(a != None and b!= None and c!= None):
        d[a, b] += freq_tri[a, b, c]

# Next word prediction
s=''
def pick_word(counter):
    "Chooses a random element."
    return random.choice(list(counter.elements()))
prefix = "he", "said"
print(" ".join(prefix))
s = " ".join(prefix)
for i in range(19):
    suffix = pick_word(d[prefix])
    s=s+' '+suffix
    print(s)
    prefix = prefix[1], suffix

```

he said

he said kotc

he said kotc made

he said kotc made profits

he said kotc made profits of

he said kotc made profits of 265

he said kotc made profits of 265 ,

he said kotc made profits of 265 , 457

he said kotc made profits of 265 , 457 vs

he said kotc made profits of 265 , 457 vs loss

he said kotc made profits of 265 , 457 vs loss eight

he said kotc made profits of 265 , 457 vs loss eight cts

he said kotc made profits of 265 , 457 vs loss eight cts net

he said kotc made profits of 265 , 457 vs loss eight cts net loss

he said kotc made profits of 265 , 457 vs loss eight cts net loss 343

he said kotc made profits of 265 , 457 vs loss eight cts net loss 343 ,

he said kotc made profits of 265 , 457 vs loss eight cts net loss 343 ,

266

he said kotc made profits of 265 , 457 vs loss eight cts net loss 343 ,

266 ,

he said kotc made profits of 265 , 457 vs loss eight cts net loss 343 ,

266 , 000

he said kotc made profits of 265 , 457 vs loss eight cts net loss

343 , 266 , 000 shares

Metrics for Language Modelings

- **Entropy:** Entropy, as a measure of the amount of information conveyed by Claude Shannon. Below is the formula for representing entropy
-

$$H(p) = \sum_x p(x) \cdot (-\log(p(x)))$$

$H(p)$ is always greater than equal to 0.

- **Cross-Entropy:** It measures the ability of the trained model to represent test data().

$$\text{data}(W_1^{i-1}).$$

$$H(p) = \sum_{i=1}^x \frac{1}{n} (-\log_2(p(w_i|w_1^{i-1})))$$

The cross-entropy is always greater than or equal to Entropy i.e the model uncertainty can be no less than the true uncertainty.

-

- **Perplexity:** Perplexity is a measure of how good a probability distribution predicts a sample. It can be understood as a measure of uncertainty. The perplexity can be calculated by cross-entropy to the exponential

$$2^{Cross-Entropy}$$

- Following is the formula for the calculation of Probability of the test set assigned by the language model, normalized by the number of words:
-

- $$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

For Example:

- Let's take an example of the sentence: '***Natural Language Processing***'. For predicting the first word, let's say the word has the following probabilities:

word	$P(\text{word} \mid \text{<start>})$
The	0.4
Processing	0.3
Natural	0.12
Language	0.18

- Now, we know the probability of getting the first word as natural.
But, what's the probability of getting the next word after getting the word '*Language*' after the word '*Natural*'.

word	$P(\text{word} \mid \text{'Natural'})$
The	0.05
Processing	0.3
Natural	0.15
Language	0.5

- After getting the probability of generating words 'Natural Language', what's the probability of getting '*Processing*'.

word	$P(\text{word} \mid \text{'Language'})$
The	0.1
Processing	0.7
Natural	0.1
Language	0.1

- Now, the perplexity can be calculated as:

$$PP(W) = \sqrt[n]{\prod_{i=1}^N \frac{1}{P(w_i|w_{i-1})}} = \sqrt[3]{\frac{1}{0.12*0.5*0.7}} \approx 2.876$$

- From that we can also calculate entropy:

$$Entropy = \log_2(2.876) = 1.524$$

- To get a better context of the text, we need higher values of n, but this will also increase computational overhead.
- The increasing value of n in n-gram can also lead to sparsity.

Types of Finite State Automation (FSA)

- **Deterministic Finite automation (DFA)**
- **Non-deterministic Finite Automation (NDFA)**

Properties of Regular Expressions

Properties of Regular Expressions

Properties of Regular Expressions

Properties of Regular Expressions

Properties of Regular Expressions

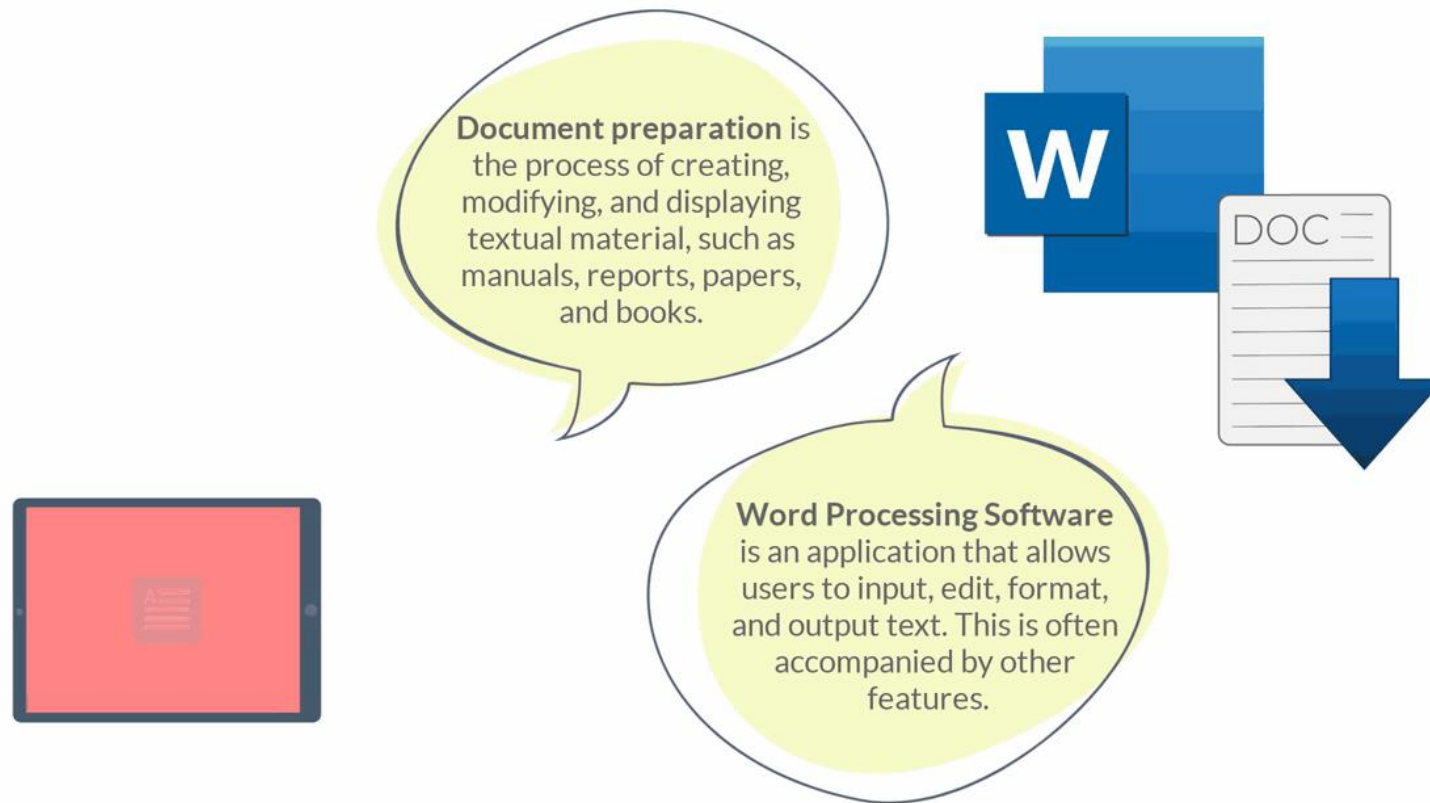
Properties of Regular Expressions

Properties of Regular Expressions

Properties of Regular Expressions

Properties of Regular Expressions

Unit-2 Word Level Processing



➤ **Types of Morphology: English and Indian Languages,**

➤ Finite-State Morphological Parsing,

➤ **building a Finite-State Lexicon,**

➤ Finite-State Transducers,

➤ FSTs for Morphological Parsing,

➤ Transducers and Orthographic rules,

➤ Word and Sentence Tokenization,

➤ Detecting and Correcting Spelling Errors,

➤ Detecting and Correcting Spelling Errors,

➤ **Minimum Edit Distance,**

➤ Human Morphological Processing

- **N-Grams: Building N-gram for spelling corrections, N-gram for language model.**
-